

# Visualization of LLL and BKZ Algorithms with Vector on Cartesian Coordinate

Muhammad Imam Chaidar Mujaddid Al-Ghiffari<sup>1[0000-1111-2222-3333]</sup> and  
Amril Syalim<sup>2,3[1111-2222-3333-4444]</sup>

Faculty of Computer Science, Universitas Indonesia

**Abstract.** The abstract should briefly summarize the contents of the paper in 150-250 words.

**Keywords:** Cryptography · Cryptanalysis · Post-Quantum · Lattice · LLL · BKZ.

## 1 Background

Cryptanalysis is the technique of analysing and breaking a ciphertext to obtain the plaintext (A. Al-Sabaawi, 2020). The history and the development of cryptanalysis itself can't be separated from the history and development of cryptography. As the cryptography techniques evolve, the cryptanalysis techniques also evolve in order to break the cryptography techniques.

One of the earliest known recorded cryptanalysis technique is the frequency analysis technique, which was used by Arab mathematician Al-Kindi in the 9th century to break monoalphabetic substitution ciphers. This technique relies on the fact that in any given language, certain letters and combinations of letters occur with varying frequencies. By analysing the frequency of letters in the ciphertext, a cryptanalyst can make deduce the likely substitutions used in the cipher.

The frequency analysis technique was arguably the most common and efficient cryptanalysis technique used for breaking classic ciphers. However, as the cipher techniques became more complex, this cryptanalysis technique became less effective. Overall, during and after World War II, mathematics became more and more crucial in both cryptography and cryptanalysis. And the modern cryptanalysis techniques are mostly based on complex mathematical concepts and algorithms, like number theory, algebra, and probability theory.

Since the most common form of modern cryptography is key-based cryptography (symmetric and asymmetric), with the onset of the development of quantum computing and subsequently quantum cryptanalysis, many commonly used modern cryptographic algorithms are threatened by quantum cryptanalysis. For example, Shor's algorithm - a quantum algorithm to finding prime factors of an integer, can efficiently factor large integers and compute discrete logarithms. Which endangers the security of commonly used asymmetric cryptographic algorithms like RSA and ECC. Not only asymmetric cryptographic

algorithms are threatened by this, Grover’s algorithm - also known as Quantum Search Algorithm, can search unsorted databases quadratically faster than classical algorithms, which threatens the security of symmetric-key cryptosystems by effectively halving their key lengths.

Which is why cryptographers around the world already started to develop cryptographic algorithms that can resist quantum cryptanalysis. These newer and stronger cryptographic algorithms are called post-quantum cryptography (PQC). Examples of PQC algorithms include Kyber and Dilithium, which are based on the hardness of lattice problems. In fact, lattice-based cryptographic algorithms dominate the PQC. The lattice-based PQC algorithms are indeed able to resist quantum computer-based cryptanalysis. For example, the quantum cryptanalysis technique of Shor’s Algorithm can’t break it efficiently.

However, just like how the historical pattern of cryptanalysis and cryptography goes in our history, as cryptographers developed lattice-based cryptographic algorithms, the cryptanalysis experts also developed lattice-based cryptanalysis techniques in order to break the lattice-based cryptographic algorithms. The most common lattice-based cryptanalysis technique is the LLL algorithm, which is a polynomial-time algorithm for finding a short and nearly orthogonal basis for a lattice. This algorithm can be used to solve the shortest vector problem (SVP) and the closest vector problem (CVP), which are both important issues in lattice-based cryptography. Apparently, for now, the most promising lattice-based cryptanalysis technique is the BKZ algorithm, which is basically the upscaled version of the LLL algorithm. Compared to LLL, BKZ can find even shorter vectors in a lattice.

The problem with these lattice-based cryptanalysis techniques is that in general, they are very complex and hard to understand, especially for people who are new to the field of cryptography and cryptanalysis. We think this is where the visualization of these algorithms can be very helpful. However, most visualizations of these algorithms don’t visualize the underlying mathematical concepts in an intuitive way, and in particular, these tools don’t show the mathematical calculations involved between input and the output. By visualizing these algorithms in an intuitive way as well showing the calculations behind the algorithms as well some simple theoretical explanations of these algorithms, we can help the users to understand these algorithms easier.

## 2 Related Works

j

### 2.1 lll-attack-runner

lll-attack-runner is a Javascript-based interactive web application for running advanced lattice basis reduction algorithms such as LLL and BKZ. It is developed by Zachary Robert Bennett with the latest update was in January 2026.

Because we aim to create a Javascript-based interactive web application for visualizing lattice-based cryptographic and cryptanalysis algorithms, in which two of them are LLL and BKZ, we will use `lll-attack-runner` as the backend for our visualization of the LLL and BKZ algorithms. However, since we want to visualize these two algorithms in an intuitive way which is easier to understand and more practical for cryptography learners, we will develop our own frontend for the visualization.

## 2.2 Herramienta de software para la visualización de la criptografía basada en retículas

Authored by Jaziel D. Flores Rodríguez and his fellow colleagues, this research paper was aimed to create a visualization tool for post-quantum cryptographic algorithms. The main focus was to graphically visualize mathematical concepts behind these post-quantum cryptographic algorithms, particularly lattice concepts which are the basis of these algorithms. The tool which was developed in this research, successfully visualizes these lattice-based post-quantum cryptographic algorithms within dimensions of 2, 3, 4, and 5. However, the LLL process for dimensions higher than 3 is still not implemented in this tool, and the BKZ algorithm is also not implemented in this tool. Which is why we will be implementing the LLL process for dimensions higher than 3 in our visualization tool, as well as implementing the BKZ algorithm which is not implemented in this tool.

## 3 Architecture

### 3.1 Backend

We're using Zachary Robert Bennett's `lll-attack-runner`, which is an interactive and Javascript-based lattice-based cryptanalysis solver, as the backend for our visualization of the LLL and BKZ algorithms. We only use three source codes from `lll-attack-runner`, which are `lll.ts`, `bkz.ts`, and `types.ts`. The `lll.ts`, per its filename, does the LLL algorithm process, and the same goes for `bkz.ts`. Both source codes need `LLLStep` interface from `types.ts`, and for `bkz.ts`, it also needs `LLRun` from `lll.ts`. Which makes sense, consider the usage of LLL algorithm in BKZ algorithm.

In each of our algorithm modules, each algorithm has its own folder, which in each of these folders, there is a single source code written in Typescript, with naming convention of the algorithm name written in lowercase. Each of these source codes are responsible for running the algorithm process which corresponds to their name. Within each of these source codes, we will use imported functions from Zachary Robert Bennett's `lll-attack-runner` source codes to run the algorithm process.

### 3.2 Frontend

For the frontend part, we will use Next.js, which is a React-based framework for building web applications, with the goal to create a user-friendly interface for our visualization of the LLL and BKZ algorithms. The user will write their own matrix (or copy-and-paste it from any sources), insert needed values (for example, delta value for both LLL and BKZ algorithm and the block size for BKZ algorithm), and the frontend will pass the values inserted by the user into the backend for the core processing. The frontend's responsibility is mainly for visualization part.

For the visualization part, we will use D3 for the visualization part, combined with React to create matrices and vectors visualizations. Our secondary objective for the frontend is to create a visualization which have animations for the steps of these algorithms. We believe the animated version of these visualizations can be understand easier by the users.

## 4 Visualization

### 4.1 Output Visualization and Textual Elements for LLL and BKZ

Visualization of both LLL and BKZ algorithms contains following five visual elements in their output section :

#### 4.1.1 Graph Panel

This is the key visual element. Psychologically speaking, human brain is usually better at understanding visual information compared to textual ones. We design the graph panel in 2D and 3D projections, both in the fashion of the Cartesian coordinate. Each vector arrows represented in this graph panel have different colour, with their vector written in the same colour as their arrow and placed above the tip of their vector arrow.

#### 4.1.2 Lovasz Status Panel

The panel shows the user of the Lovasz status of the current basis. If the current basis fulfill the Lovasz condition, it will show the user a green check mark and "Satisfied" text, and if not, a red cross mark and "Not Satisfied" text will be shown to the user.

#### 4.1.3 Step and Calculation Details

This panel will tell the user of the current step (for example, "Basis size reduction"). The user can see the details of the calculation in the step by clicking "Show Calculations" button at the upper right corner of this panel.

#### 4.1.4 Current Basis Panel

Like its namesake, this panel's sole function is to show the user the current basis.

#### 4.1.5 Shortest Vector Panel (BKZ only)

Only for BKZ algorithm, this panel's sole function is to inform the user the shortest vectors found in the current basis. Only shown in the last step.

### 4.2 Visualization of the LLL Algorithm

First, we have the LLL algorithm. The input panel for this algorithm has matrix and delta value insertion fields, and located in the left half of the page. After the user inserted matrix and delta value properly, and clicked "Process Matrix" button, user can see the "Initial Matrix" as well "Initial Basis" panels below the input panel. To the right half of the page, there is the output section, contains four of five visual and textual information panels, without Shortest Vector Panel.

User can click "Back" and "Next" button in the upper part of the output section to see the previous and next steps of the LLL algorithm process. All visual and textual information panels in the output section are subjects of change according to the current step of the LLL algorithm process.

### 4.3 Visualization of the BKZ Algorithm

Next, we have the BKZ algorithm, which is one of the more powerful lattice reduction algorithm. Because theoretically this algorithm could be comprised of multiple LLL processes, we design the output section to show the user the number of LLL processes as well an additional information called "Current Process", which provides the user of the steps that is unique to BKZ algorithm, which are the current number of the LLL process and SVP enumeration.

The only differences with LLL page is in the input panel, which has an additional, mandatory field for the block size, and in the output section, in which has Shortest Vector panel. Moreover, in the upper part of the output section, we place four additional information, which are "Block Size", "Delta", "Total LLL Processes", and "Current Process". Out of all of these additional information, only "Current Process" isn't static.

Aside from additional panels and input field, the rest of the visualization and basically mechanism of the output's UI is the same as the LLL ones.

## 5 Result

Please note that the first paragraph of a section or subsection is not indented. The first paragraph that follows a table, figure, equation etc. does not need an indent, either.

Subsequent paragraphs, however, are indented.

### 5.1 LLL Visualization

### 5.2 BKZ Visualization

*5.2.0.1 Sample Heading (Fourth Level)* The contribution should contain no more than four levels of headings. Table 1 gives a summary of all heading levels.

**Table 1.** Table captions should be placed above the tables.

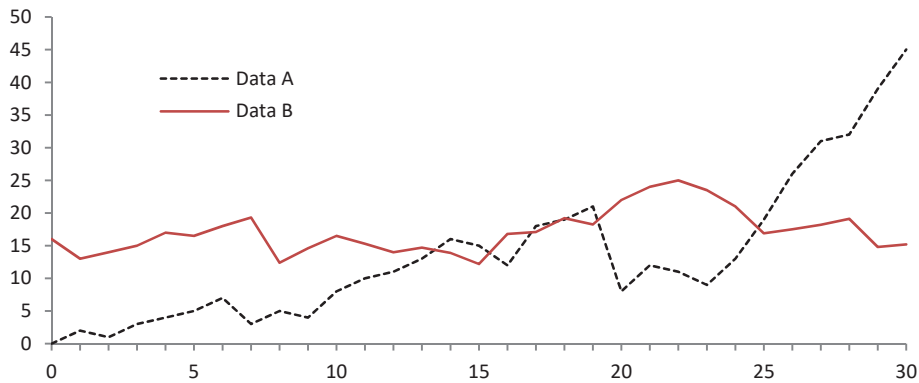
| Heading level     | Example                                     | Font size and style |
|-------------------|---------------------------------------------|---------------------|
| Title (centered)  | <b>Lecture Notes</b>                        | 14 point, bold      |
| 1st-level heading | <b>1 Introduction</b>                       | 12 point, bold      |
| 2nd-level heading | <b>2.1 Printing Area</b>                    | 10 point, bold      |
| 3rd-level heading | <b>Run-in Heading in Bold.</b> Text follows | 10 point, bold      |
| 4th-level heading | <i>Lowest Level Heading.</i> Text follows   | 10 point, italic    |

Displayed equations are centered and set on a separate line.

$$x + y = z$$

(1)

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 1).



**Fig. 1.** A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

**Theorem 1.** *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

*Proof.* Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [1], an LNCS chapter [2], a book [3], proceedings without editors [4], and a homepage [5]. Multiple citations are grouped [1–3], [1, 3–5].

**5.2.1 Acknowledgments.** A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

**5.2.2 Disclosure of Interests.** It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments<sup>1</sup>, for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

## References

1. Author, F.: Article title. *Journal* **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) *CONFERENCE 2016, LNCS*, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.1007/1234567890>
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: *9th International Proceedings on Proceedings*, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25

---

<sup>1</sup> If EquinOCS, our proceedings submission system, is used, then the disclaimer can be provided directly in the system.